CS 301
High-Performance Computing

Lab 02 - Performance Analysis of Matrix Multiplication

Prasham Shah (202301252)
Dhruvi Jobanputra (202301401)

February 17, 2026

Contents

1 Introduction **3**

2 Problem Description **3**

3 Implemented Algorithms **3**
3.1 Conventional Matrix Multiplication . . . 3
3.2 Matrix Multiplication Using Transpose . . . 3
3.3 Blocked Matrix Multiplication . . . 3

4 Experimental Setup **4**
4.1 Hardware . . . 4
4.2 Software . . . 4

5 Performance Metrics **4**

6 Results **5**
6.1 Execution Time vs Matrix Size . . . 5
6.2 Performance (MFLOPS) vs Matrix Size . . . 7
6.3 Execution Time vs Matrix Size . . . 9
6.4 Performance (MFLOPS) vs Matrix Size . . . 11
6.5 Execution Time vs Matrix Size . . . 13
6.6 Performance (MFLOPS) vs Matrix Size . . . 15

7 Results **17**
7.1 Execution Time vs Matrix Size . . . 17
7.2 Performance (MFLOPS) vs Matrix Size . . . 19

8 Results **21**
8.1 Execution Time vs Matrix Size . . . 21
8.2 Performance (MFLOPS) vs Matrix Size . . . 23

9 Discussion **24**

10 Conclusion **24**

Abstract

This experiment analyzes the performance of serial matrix multiplication on a local workstation and an HPC cluster. Different implementations, including conventional loop ordering, transposed matrix multiplication, and blocked matrix multiplication, are evaluated. The impact of memory access patterns and cache utilization on execution time and performance is studied by varying the matrix size.

1 Introduction

Matrix multiplication is a fundamental operation in scientific computing and many real-world applications. Although its computational complexity remains the same for different implementations, execution time can vary significantly due to differences in memory access patterns and cache behavior.

In this lab, we study the performance of serial matrix multiplication by:

- Interchanging loop orders
- Using matrix transpose for improved locality
- Applying block (tiled) matrix multiplication

Experiments are performed on both a local system and an HPC cluster to compare performance trends.

2 Problem Description

Given two square matrices A and B of size $N \times N$, the goal is to compute:

$$C = A \times B$$

The matrix size is varied as:

$$N = 2^1, 2^2, \ldots, 2^{12}$$

All computations are performed using double precision floating-point arithmetic.

3 Implemented Algorithms

3.1 Conventional Matrix Multiplication

The standard matrix multiplication is implemented using three nested loops:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \times B[k][j]$$

This implementation serves as the baseline for performance comparison.

3.2 Matrix Multiplication Using Transpose

In this approach, matrix B is transposed before multiplication. This improves spatial locality by accessing contiguous memory locations in the inner loop.

3.3 Blocked Matrix Multiplication

Blocked matrix multiplication divides matrices into smaller submatrices of size $B \times B$. The block size is chosen as:

$$B = \min(32, N)$$

This technique improves cache reuse and reduces memory access latency.

4 Experimental Setup

4.1 Hardware

- **Local Machine:** <CPU model, cache size>
- **HPC Cluster:** <CPU model, cache hierarchy>

4.2 Software

- Compiler: `g++`
- Optimization Flag: `-O3`
- Timing Function: `clock_gettime(CLOCK_MONOTONIC)`

Each experiment is repeated 5 times and the average execution time is recorded.

5 Performance Metrics

The total number of floating-point operations for matrix multiplication is:

$$\text{FLOPs} = 2N^3$$

The performance in MFLOPS is calculated as:

$$\text{MFLOPS} = \frac{2N^3}{\text{Execution Time} \times 10^6}$$

6 Results

Conventional matrix multiplication

6.1 Execution Time vs Matrix Size

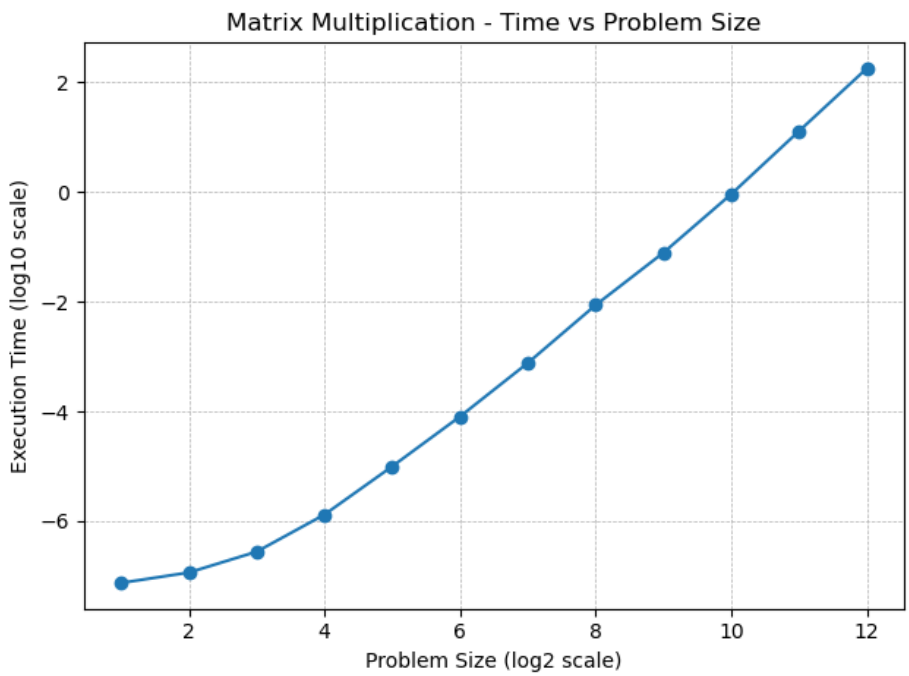

Figure 1: Execution Time vs Matrix Size (Local Machine)

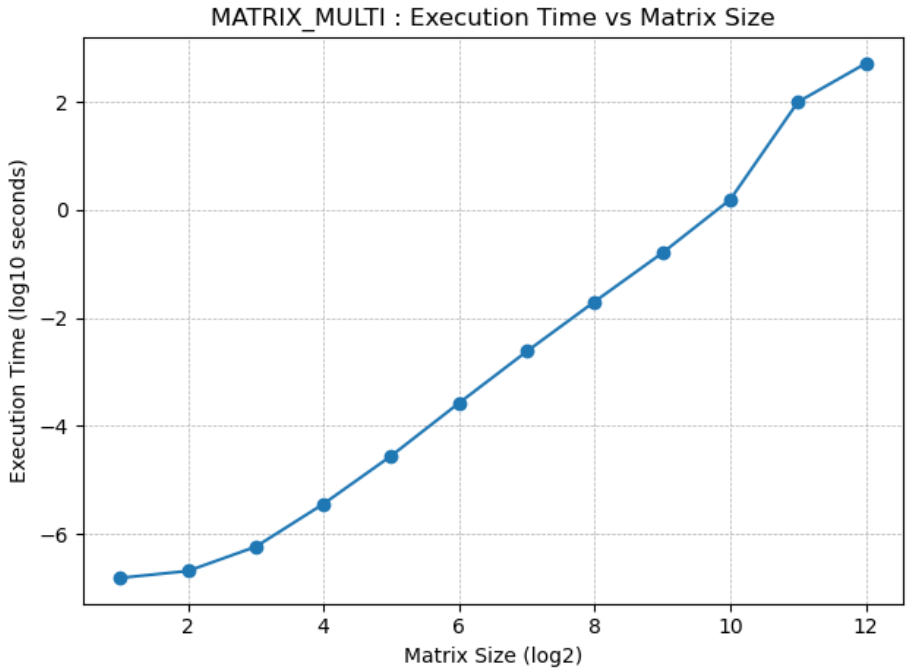

Figure 2: Execution Time vs Matrix Size (HPC Cluster)

6.2 Performance (MFLOPS) vs Matrix Size

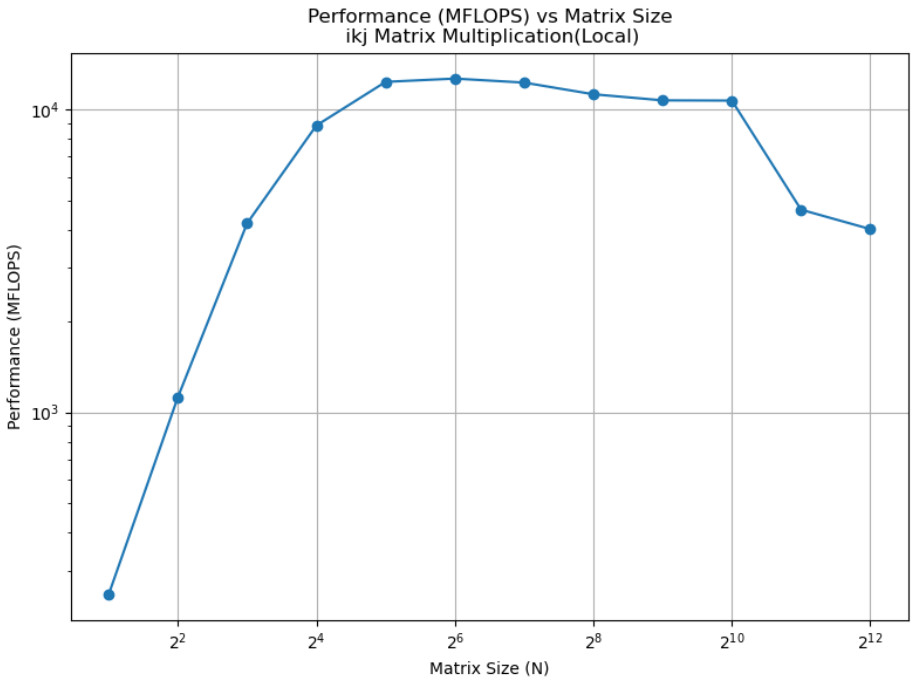

Figure 3: MFLOPS vs Matrix Size (Local Machine)

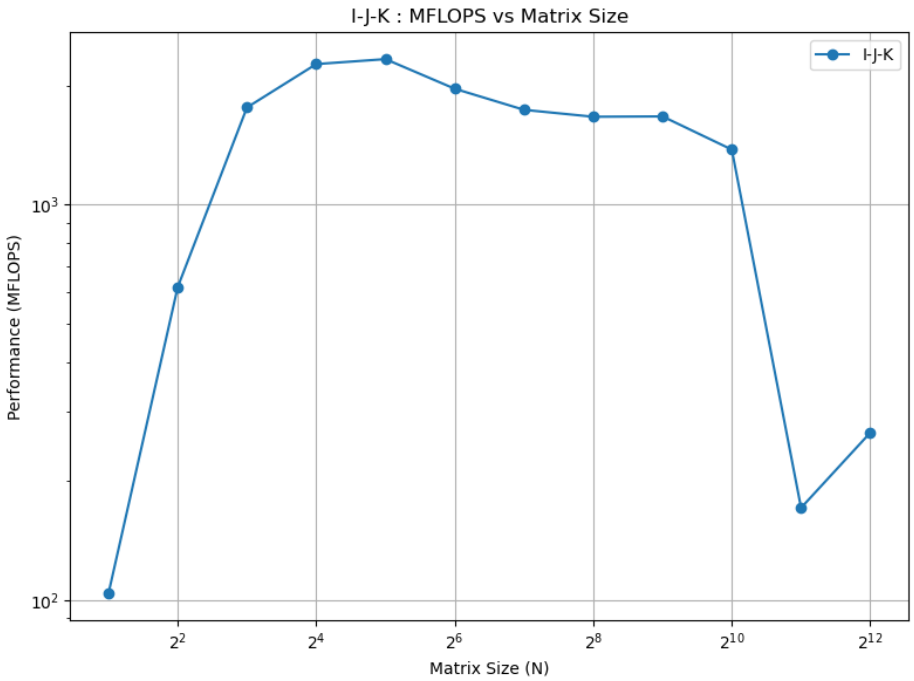

Figure 4: MFLOPS vs Matrix Size (HPC Cluster)

Conventional matrix multiplication - ikj

6.3 Execution Time vs Matrix Size

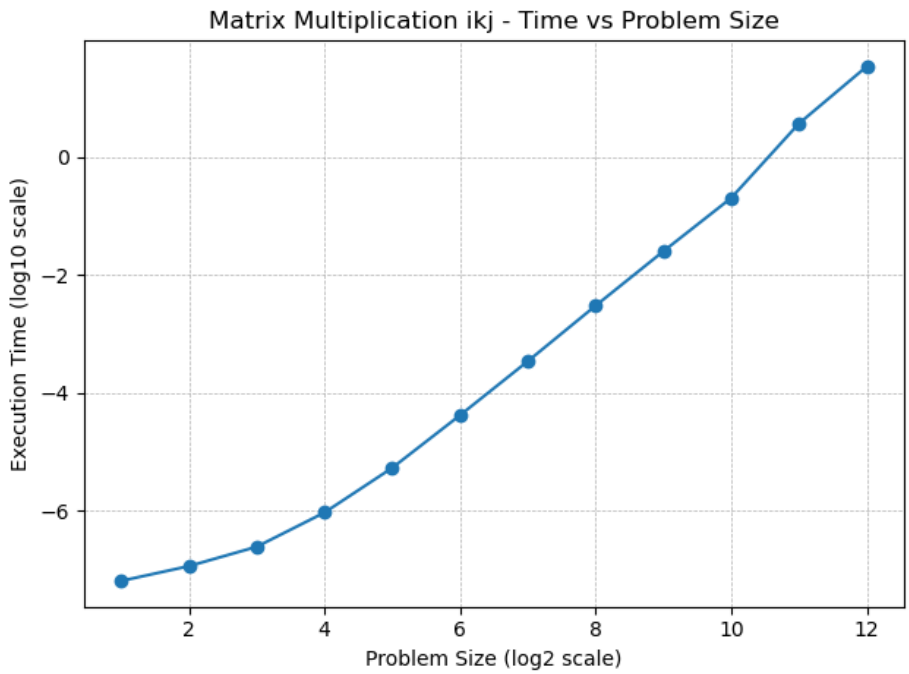

Figure 5: Execution Time vs Matrix Size (Local Machine)

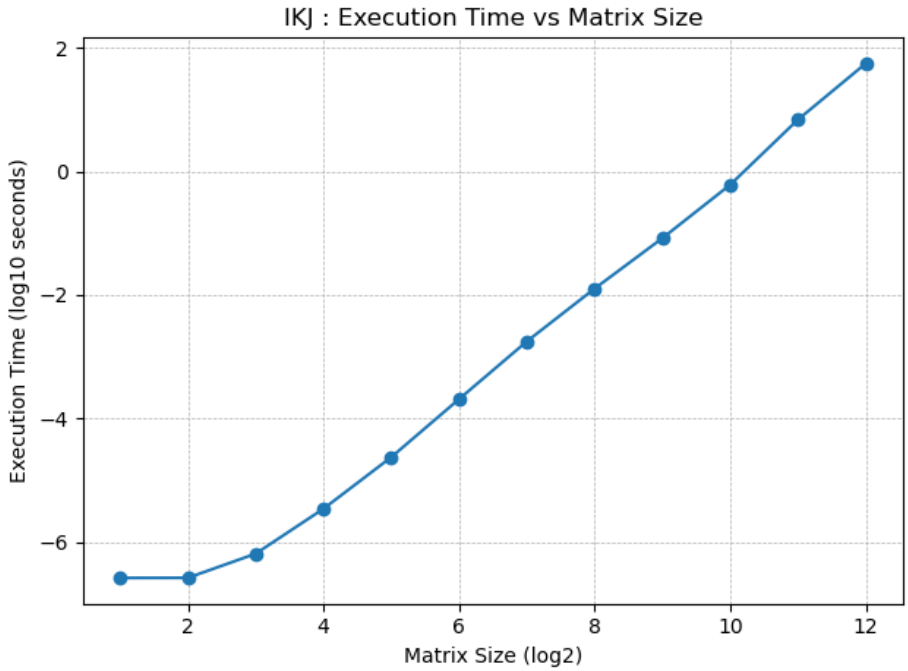

Figure 6: Execution Time vs Matrix Size (HPC Cluster)

6.4 Performance (MFLOPS) vs Matrix Size

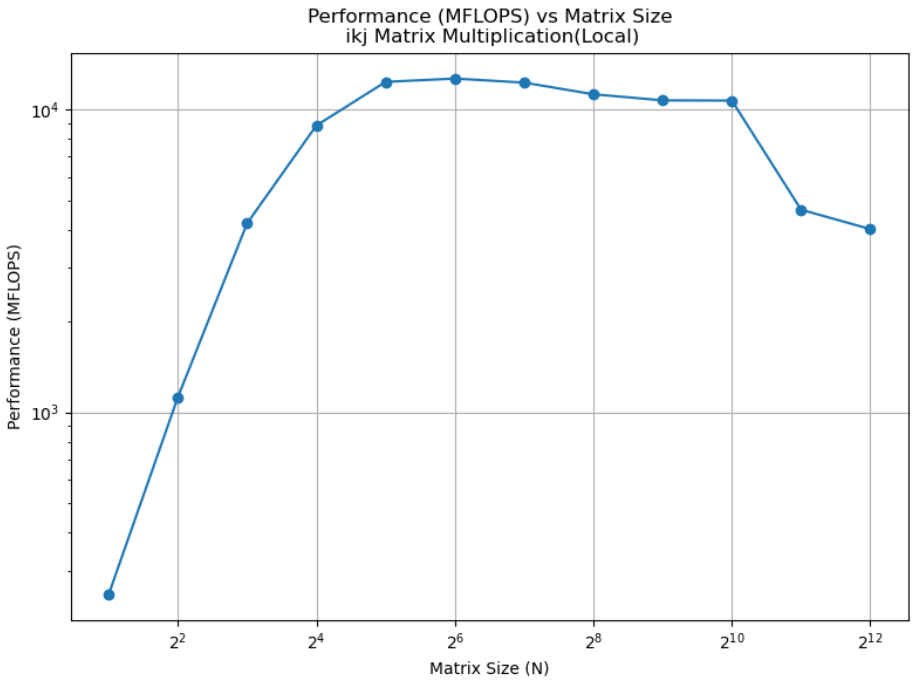

Figure 7: MFLOPS vs Matrix Size (Local Machine)

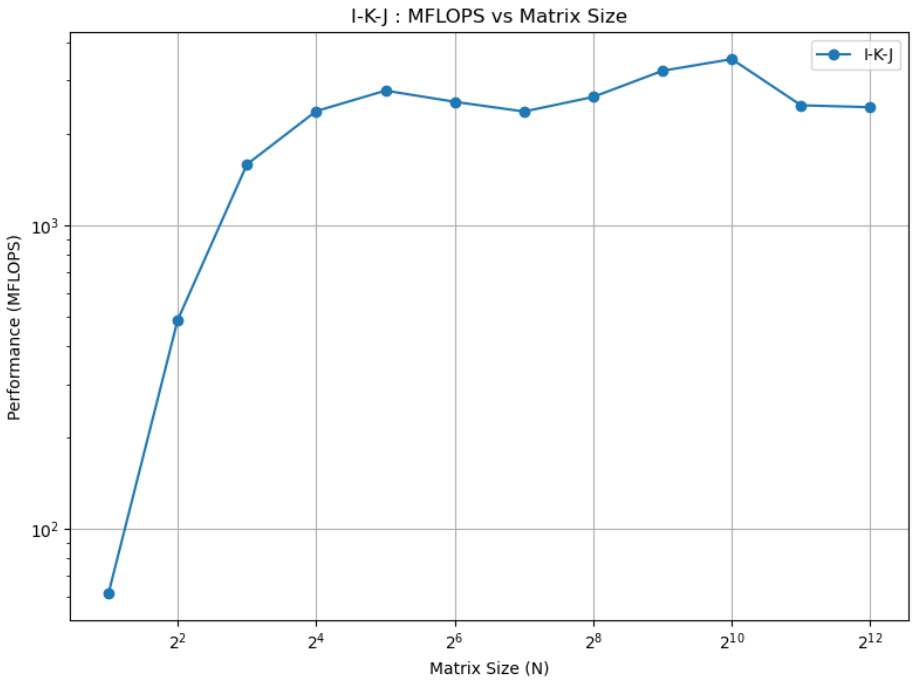

Figure 8: MFLOPS vs Matrix Size (HPC Cluster)

Conventional matrix multiplication - kij

6.5 Execution Time vs Matrix Size

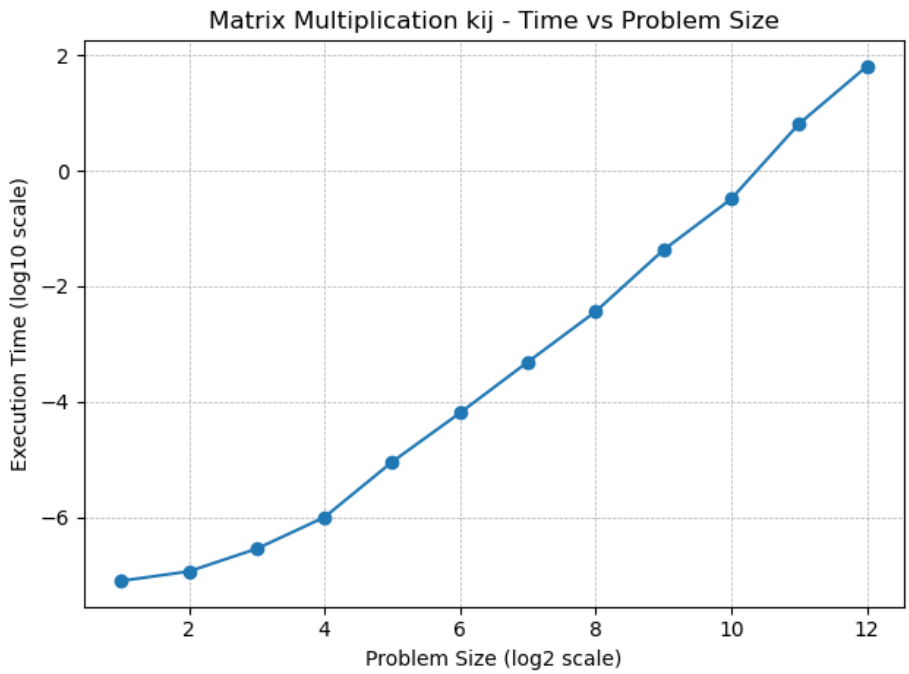

Figure 9: Execution Time vs Matrix Size (Local Machine)

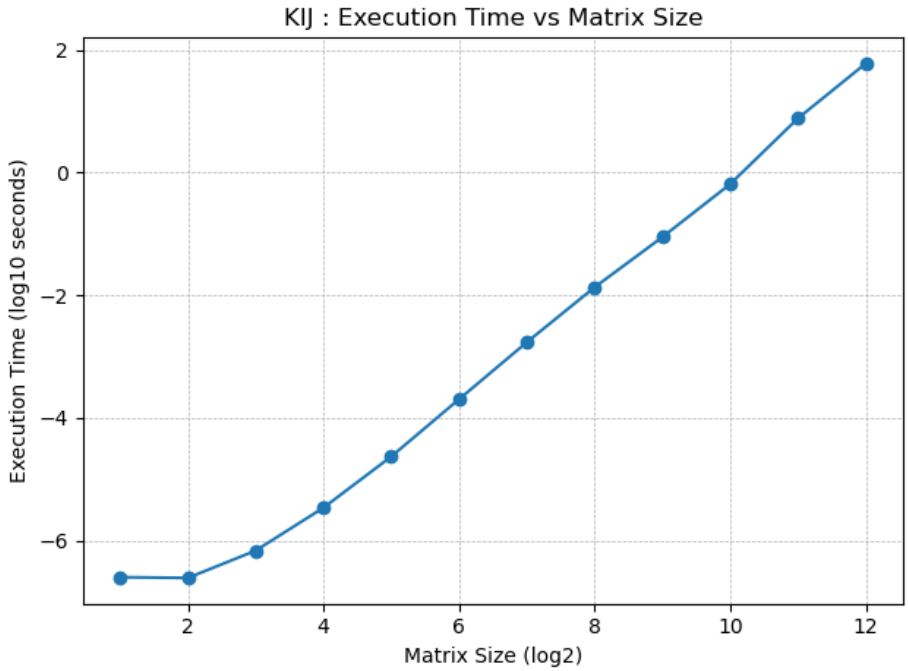

Figure 10: Execution Time vs Matrix Size (HPC Cluster)

6.6 Performance (MFLOPS) vs Matrix Size

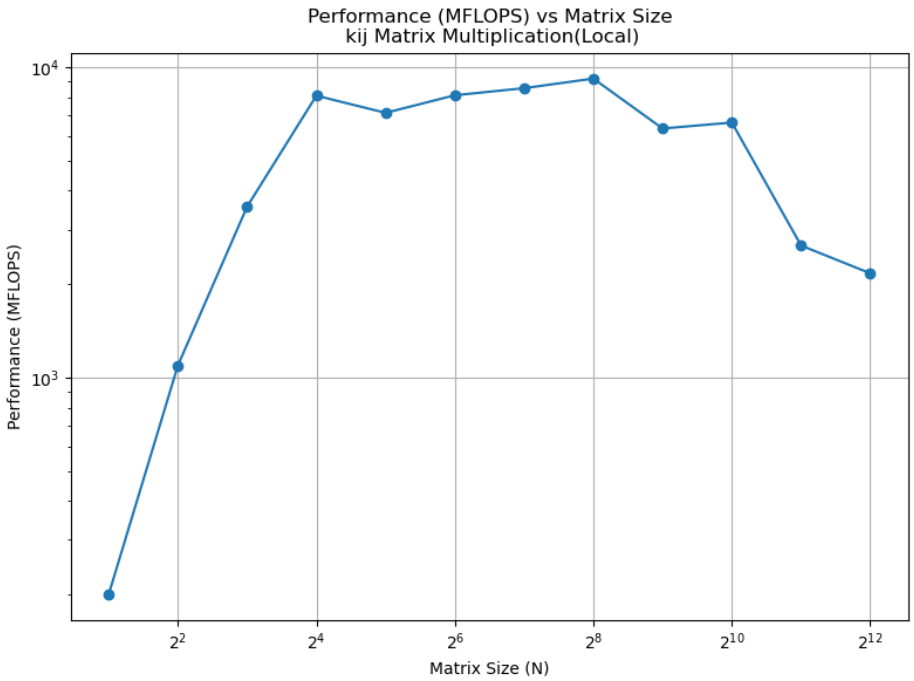

Figure 11: MFLOPS vs Matrix Size (Local Machine)

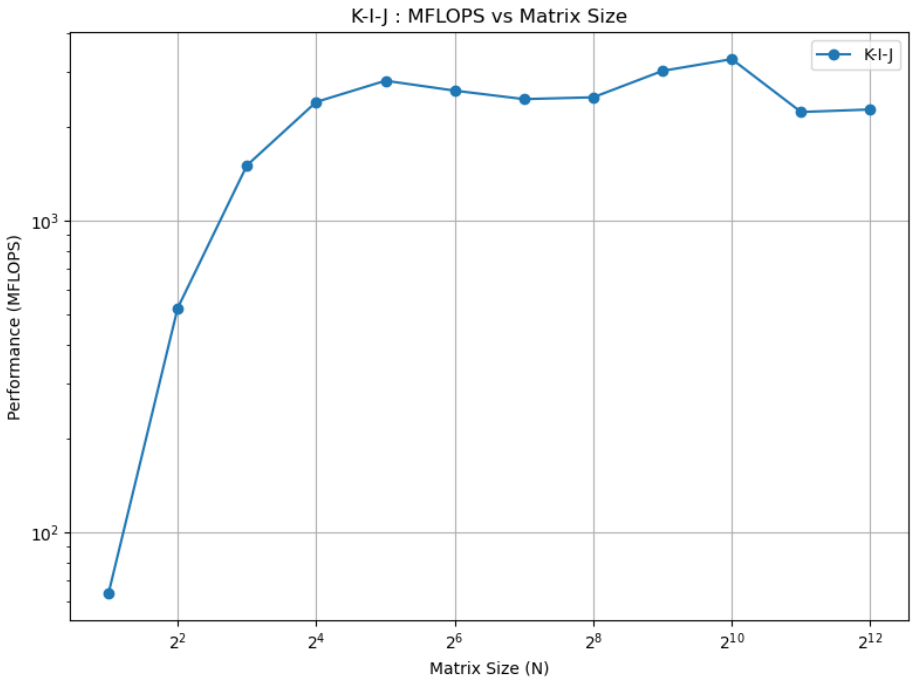

Figure 12: MFLOPS vs Matrix Size (HPC Cluster)

7 Results

Matrix multiplication using transpose

7.1 Execution Time vs Matrix Size

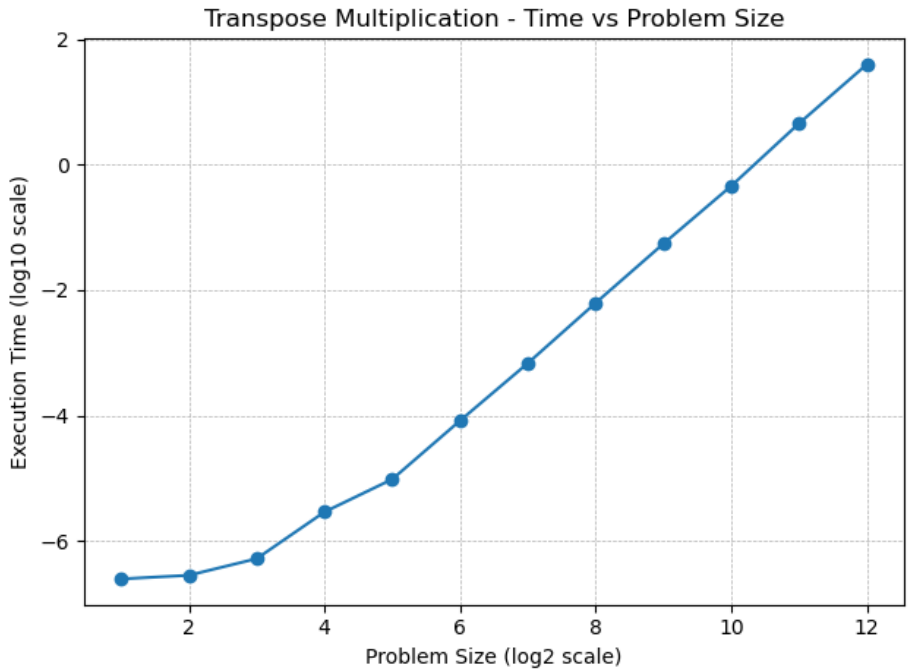

Figure 13: Execution Time vs Matrix Size (Local Machine)

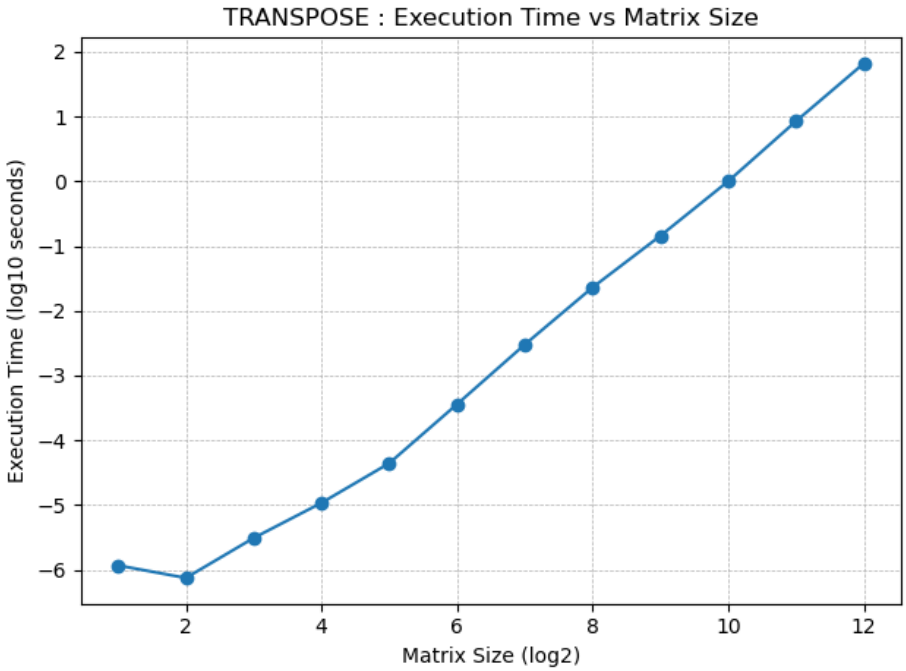

Figure 14: Execution Time vs Matrix Size (HPC Cluster)

7.2 Performance (MFLOPS) vs Matrix Size

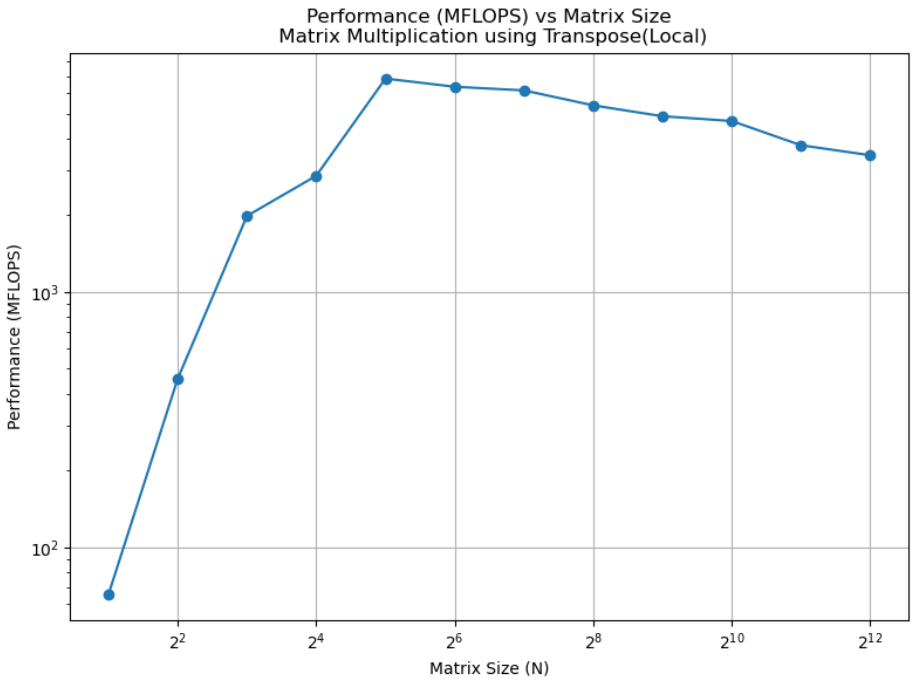

Figure 15: MFLOPS vs Matrix Size (Local Machine)

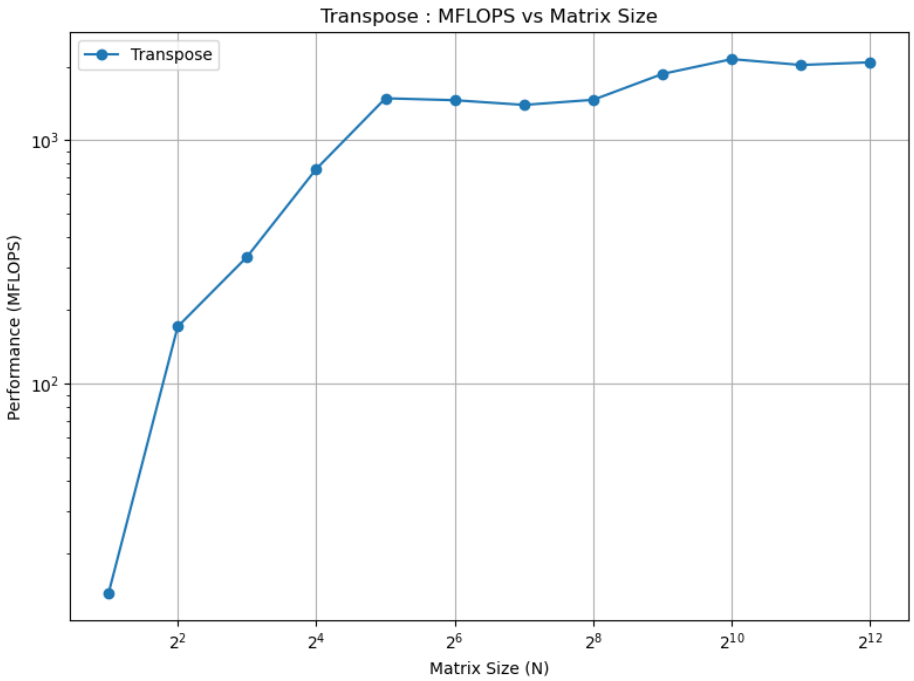

Figure 16: MFLOPS vs Matrix Size (HPC Cluster)

8 Results

Block matrix multiplication

8.1 Execution Time vs Matrix Size

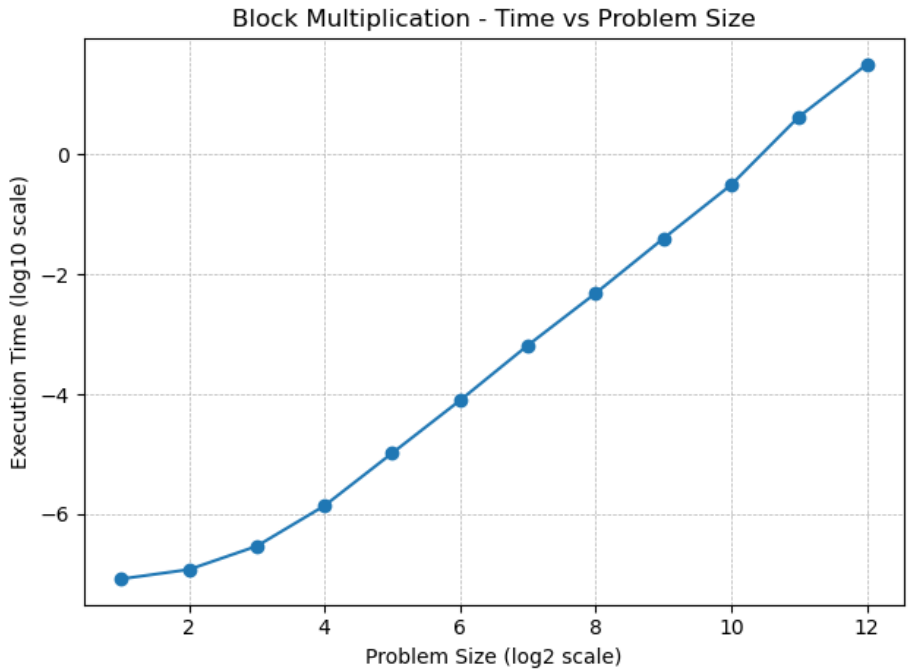

Figure 17: Execution Time vs Matrix Size (Local Machine)

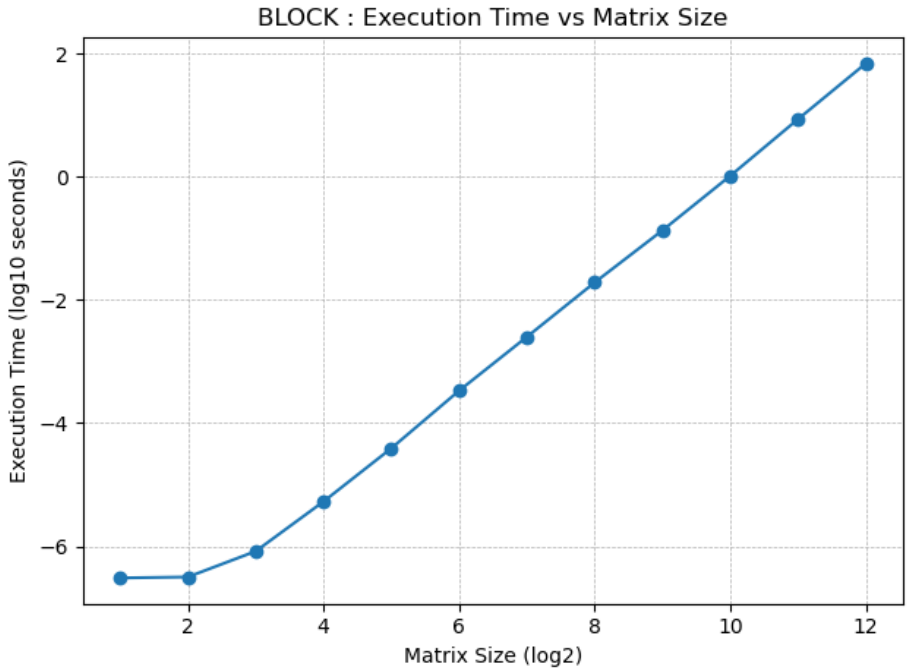

Figure 18: Execution Time vs Matrix Size (HPC Cluster)

8.2 Performance (MFLOPS) vs Matrix Size

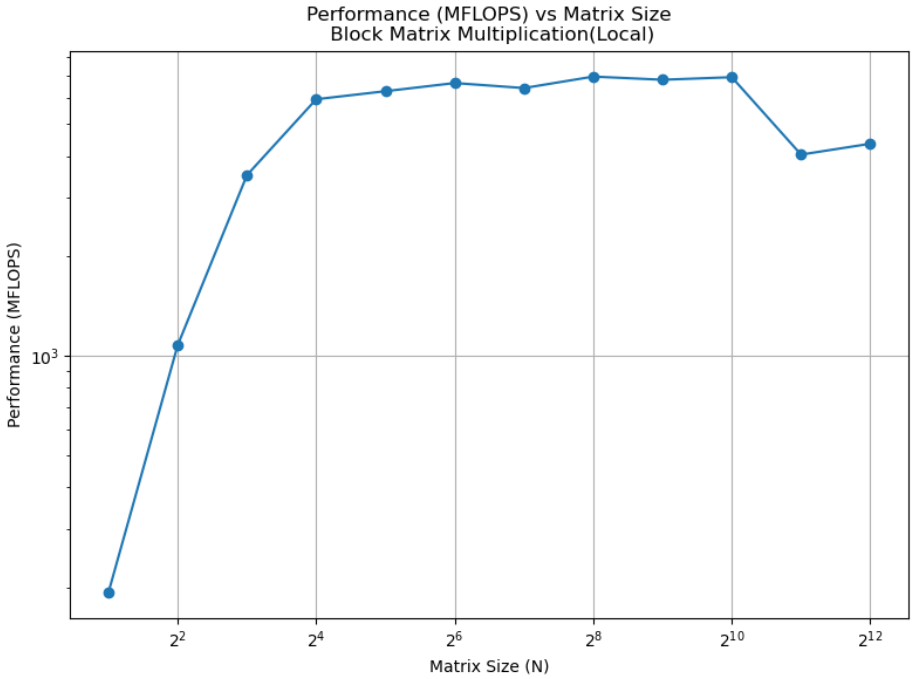

Figure 19: MFLOPS vs Matrix Size (Local Machine)

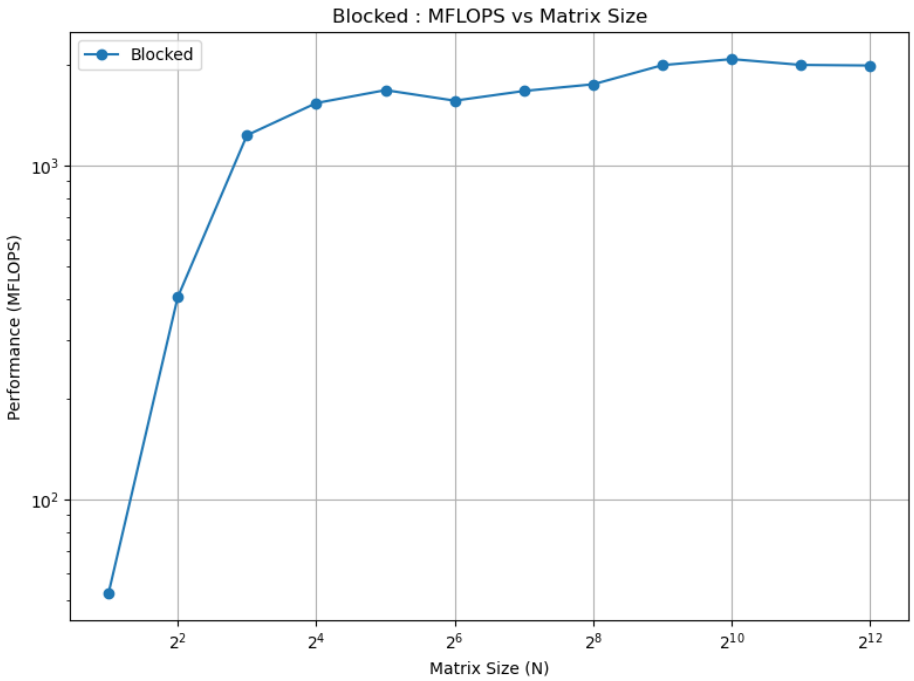

Figure 20: MFLOPS vs Matrix Size (HPC Cluster)

9 Discussion

Matrix multiplication performance varied significantly with loop ordering and memory access patterns. The baseline I–J–K implementation showed the poorest performance due to non-contiguous access of matrix B, causing frequent cache misses. The loop-interchanged variants I–K–J and K–I–J improved performance by accessing matrix B more contiguously and increasing cache reuse, with I–K–J generally performing slightly better.

Transpose-based multiplication further improved locality by converting column accesses into row accesses. Blocked multiplication achieved the best performance by operating on cache-sized submatrices and maximizing data reuse. Performance degradation for large matrix sizes occurred when data exceeded cache capacity. The HPC cluster consistently outperformed the local machine due to its larger cache and higher memory bandwidth.

10 Conclusion

This lab shows that loop ordering and cache-aware optimizations strongly affect matrix multiplication performance. The naive I–J–K ordering performed worst, while loop interchange (I–K–J and K–I–J) and transpose-based access improved locality and reduced cache misses. Blocked multiplication provided the highest performance by maximizing cache utilization. The results confirm that memory hierarchy plays a critical role in high-performance computing, and systems with larger caches and higher bandwidth, such as the HPC cluster, achieve better performance.

References

- HPC Lab Manual
- Lecture Notes on Memory Hierarchy and Cache Optimization